

Pollinator Habitat — Developer Overview

This document gives a topological map of the entire application: what it is, how its three apps relate to each other, what each page does, and what each API endpoint does. Use it as the starting point; the deep-dives are in the linked docs.

Deeper Documentation

Document	What it covers
backend-flow.md	Every layer of the backend — startup, routing, middleware, handlers, services, Docker
frontend-flow.md	Every frontend page and component line-by-line — state, API calls, rendering logic
PORTAL_FLOW.md	In-depth portal guide — page state machine, components, fetch services, env config, testing setup
API_SPECIFICATION.md	All 34 endpoints — exact request/response schemas, auth, error codes
BACKEND_SERVICES.md	Service layer, utilities, and all DB operation classes
DATABASE_SCHEMA.md	7-table schema, ER relationships, migration history, seed data
SHARED_TYPES.md	Route/RouteNode/FactNode interfaces, spritesheet coordinate system
ACCESSIBILITY.md	localStorage schema, High Contrast CSS, TTS setting and implementation
DOCKER_SETUP.md	Docker onboarding — all 4 compose files, all Dockerfiles, Nginx, env vars, troubleshooting
ADDING_ROUTES.md	How to add pollinator routes, change spritesheet images, re-seed the DB
TEST_DOCUMENTATION.md	Every test file explained — what it tests and why, test-by-test
ROUTE_STATISTICS_SPEC.md	Spec for the Route Statistics feature — 10 metrics, 20 endpoints, API contract, portal UI design
NEXT_ROUTE_FLOW.md	Deep trace of the "Next" button flow — route deduplication logic, cycle algorithm, concurrency handling
MYSQL_LICENSE_ISSUE.md	Why MySQL's GPL license is a problem and how the included PostgreSQL stack resolves it

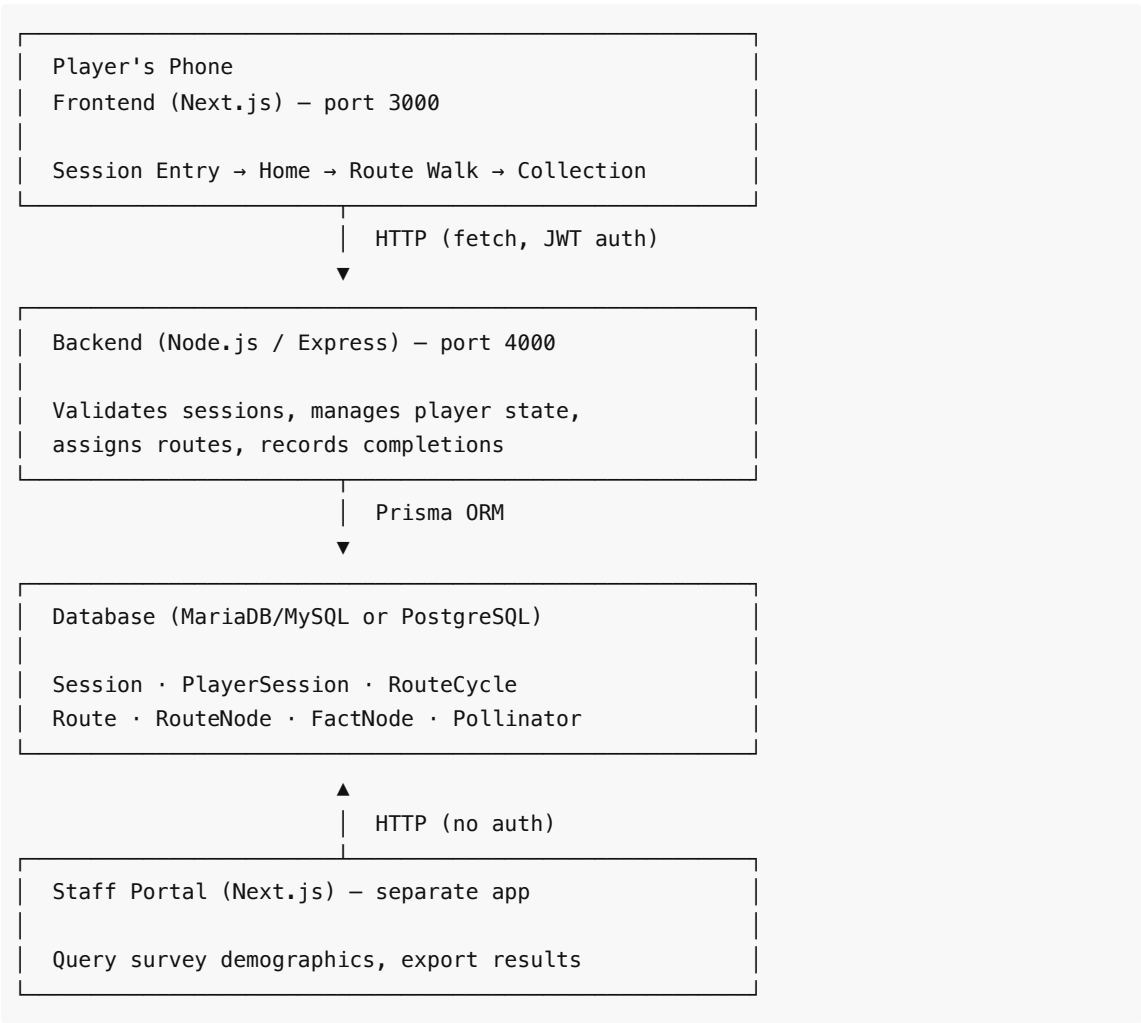
What Is This App?

Pollinator Habitat is a **nature walk game** for visitors at a park or nature site. Players walk a physical route, stop at marked locations, and learn facts about pollinators along the way. The experience is delivered

entirely through a web app on their phone — no app store install required.

A **staff portal** runs separately and lets staff query survey data collected during sessions (party sizes, demographics).

System Topology



The frontend and portal both talk to the same backend. The frontend uses JWT authentication; the portal does not (it is intended for internal staff use only).

The Three Apps

1. Frontend (frontend/)

The player-facing game. Runs on port 3000. Built with Next.js (React). Deployed as a Docker container.

Players access it at the park on their own devices via a shared URL or QR code.

2. Backend (backend/)

The Express API server. Runs on port 4000. Built with Node.js + TypeScript + Prisma. Deployed as a Docker container.

Handles all game logic: session validation, player registration, route assignment, and route completion. All mutable state lives here (in the database).

3. Portal (`portal/`)

The staff dashboard. A separate Next.js app. Staff use it to search sessions and view survey data about who attended. Not player-facing.

Database Schema — Key Tables

Table	Purpose
Session	Represents one event/day. Has a <code>sessionDate</code> and a numeric ID staff share with players
PlayerSession	One row per player per session. Created on <code>/api/start-game</code> . Holds party size
RouteCycle	One row per route assignment. Created each time a player is given a route. <code>wasCompleted</code> , <code>cycleNumber</code> , <code>completionMs</code>
Route	A named walking route (e.g. "Bat", "Human"). Has <code>RouteNode</code> children
RouteNode	A stop on a route — an index card location
FactNode	A pollinator fact tied to a route node
Pollinator	A pollinator species — name matches the route name and sprite sheet key

`Session` → `PlayerSession` → `RouteCycle` is the core game-state chain. Everything else is reference data.

Frontend Pages

`/` — Session Entry

The landing page. Players type a 9-digit session code. On submit, the app calls `POST /api/` to validate the session and receive a JWT token. The token is stored in `localStorage` and used for all subsequent authenticated requests.

After a successful validation, a button appears to navigate to `/home` .

`/home` — Home Menu

The main menu. Shows buttons to start a route, view the pollinator collection, or change accessibility settings. Displays a live count of pollinators discovered vs. the total available.

On mount, calls `POST /api/get-pollinator-names` (authenticated) to get the total pollinator count. Completed routes are read from `localStorage` via `getCompletedRoutes()` . Clears any replay state via `setReplayRoute(null)` on load.

`/route` — Route Walk (main game screen)

The core game loop. On load, calls `POST /api/start-game` to get (or resume) the player's currently assigned route. The route contains a list of stops (`routeNodes`) and associated facts (`factNodes`).

The player taps through each stop, reads facts at each location, and sees a pollinator reveal at the final stop. A party size survey (`PartySizeSurveyPopUp`) appears after the first route completion and submits to `POST /api/add-children-adults` .

After completing a route, calls `POST /api/complete-route` to mark it done and receive the next assigned route. If no more routes remain, the player is shown a final completion screen.

TTS (text-to-speech) is available via a `TTSButton` component on each card. All speech uses `rate=0.75` for clarity.

→ See [frontend-flow.md](#) for the full state machine and rendering details.

`/pollinator-collection` — Discovered Pollinators

Displays discovered pollinators with a replay button, and undiscovered pollinators as black silhouettes. Reads completed routes from `localStorage` and calls `POST /api/get-pollinator-names` (authenticated) to get all pollinators and their spritesheet coordinates. Incomplete routes are derived by comparing the API list against what's in `localStorage`. Updates live via `storage` and `completedRoutesUpdated` events.

`/accessibility` — Accessibility Settings

Lets players toggle accessibility features. Settings are stored in `localStorage` only — nothing is sent to the backend.

Available settings:

- **Text-to-Speech (auto-read):** reads each card aloud automatically on navigation
- **High Contrast Mode:** applies a CSS class to increase visual contrast

`buttonPressReadAloud` and `autoReadAloud` are stored as `localStorage` keys but have no UI controls on this page — they are internal flags only.

`/redirecting` — Countdown Redirect

Shown when a user navigates to an unknown URL (caught by the middleware allowlist). Counts down and redirects back to `/` .

Backend API Endpoints

All routes are `POST` unless noted. Base path: `http://localhost:4000` .

Game Routes (player-facing, JWT required where noted)

Method	Path	Auth	What it does
POST	<code>/api/</code>	No	Validates a session code. Returns a signed JWT if the session exists

POST	/api/start-game	JWT	Gets or creates a <code>PlayerSession</code> for this player, returns their current active route (or assigns a new one). Returns a <code>RouteDTO</code>
POST	/api/complete-route	JWT	Marks the player's current <code>RouteCycle</code> as complete, picks and assigns the next route, returns updated state: <code>routesCompleted</code> , <code>newActiveRoute</code> , <code>completedPollinators</code>
POST	/api/create-session	No	Creates a new <code>Session</code> record. Used by staff to set up a new event day
POST	/api/add-children-adults	JWT	Saves the party size survey response (number of adults and children) to the player's <code>PlayerSession</code>
POST	/api/get-pollinator-names	JWT	Returns all pollinators from the DB as <code>{ pollinator: [{ name, coord }] }</code> — each entry is one route's final node. Used by home and collection pages.
GET	/api/health	No	Returns <code>{ status: "ok" }</code> . Used by Docker health checks

Portal / Stats Routes (staff-facing, no auth)

These endpoints power the staff portal's survey data views. All accept either a `sessionId` or date range parameters.

Method	Path	What it returns
POST	/api/get-children-adults	Party size breakdown for one session
POST	/api/get-sessions-playerids-by-child-size	Sessions filtered by number of children
POST	/api/get-sessions-playerids-by-family-size	Sessions filtered by total family size
POST	/api/get-sessions-playerids-by-adult-size	Sessions filtered by number of adults
POST	/api/get-child-adult-data-by-start-end-date	Party size data for a date range (start → end)
POST	/api/get-child-adult-data-to-forever	Party size data from a start date forward
POST	/api/get-child-adult-data-to-end-date	Party size data up to an end date

Route Statistics Routes (staff-facing, no auth)

20 endpoints at `/api/route-stats/*` — 10 metrics × 2 scope modes (session and date-range). All implemented. See [ROUTE STATISTICS SPEC.md](#) §5 for the full endpoint table and response shapes.

Metric	Session endpoint	Date-range endpoint
Total routes completed	<code>/api/route-stats/total-routes-completed-by-session</code>	<code>/api/route-stats/total-routes-completed-by-date-range</code>

Total connected users	/api/route-stats/total-connected-users-by-session	/api/route-stats/total-connected-users-by-date-range
Users started >1 route	/api/route-stats/users-started-more-than-one-route-by-session	/api/route-stats/users-started-more-than-one-route-by-date-range
Users completed >1 route	/api/route-stats/users-completed-more-than-one-route-by-session	/api/route-stats/users-completed-more-than-one-route-by-date-range
Users never started	/api/route-stats/users-never-started-route-by-session	/api/route-stats/users-never-started-route-by-date-range
Started but not finished	/api/route-stats/users-started-but-not-finished-by-session	/api/route-stats/users-started-but-not-finished-by-date-range
Total replays	/api/route-stats/total-replays-by-session	/api/route-stats/total-replays-by-date-range
Avg routes / user	/api/route-stats/avg-routes-per-user-by-session	/api/route-stats/avg-routes-per-user-by-date-range
Avg routes / session	/api/route-stats/avg-routes-per-session-by-session	/api/route-stats/avg-routes-per-session-by-date-range
Replay ratio	/api/route-stats/replay-ratio-by-session	/api/route-stats/replay-ratio-by-date-range

→ See [backend-flow.md](#) for the full handler implementations and service layer details.

→ See [API SPECIFICATION.md](#) for exact request/response schemas.

The Game Loop (end-to-end)

1. Staff creates a Session (POST /api/create-session)
 - Session ID is shared with players (on a sign or QR code)
2. Player enters session code on "/"
 - POST /api/ – validates session, receives JWT
 - JWT stored in localStorage under "authToken"
3. Player taps "Start Route" on "/home"
 - Navigates to "/route"
4. On load, "/route" calls POST /api/start-game
 - Backend: getOrCreatePlayerSession → getActiveRoute (or allocateNew)
 - Returns RouteDTO: { routeId, routeNodes[], factNodes[] }
5. Player walks the route, taps through each stop and fact card
6. At the last fact card, PartySizeSurveyPopUp appears (first route only)
 - Player fills in party size → POST /api/add-children-adults
7. Player taps "Complete Route"

```
→ POST /api/complete-route
→ Backend: marks RouteCycle.wasCompleted=true, picks next route
→ Returns: { routesCompleted, newActiveRoute, completedPollinators }
```

8. If `newActiveRoute` is non-null → player is shown next route
If `newActiveRoute` is null → player is shown final completion screen

9. Player can visit `"/pollinator-collection"` to see what they've found

JWT Flow

`POST /api/` validates the session and returns a JWT signed with `JWT_SECRET`. The token contains the `sessionId` as a claim.

Protected routes extract the `sessionId` from the token (not from the request body) — this prevents clients from claiming to be in a different session. They also accept an optional `playerId` in the request body to resume an existing player identity across device refreshes.

The frontend stores the token in two places:

- `localStorage["authToken"]` — written by `page.tsx` directly
- `localStorage["pollinator_jwt_v1"]` — written by `jwtService.ts` (used for all authenticated calls through the service layer)

Both keys are set at validation time. If only one is present, some calls may fail — both must be set for the full game flow to work.

Portal (Staff Dashboard)

The portal is a Next.js app with five pages: a splash/auth screen (`page.tsx`), a navigation menu (`menu/page.tsx`), a survey data search tool (`survey/page.tsx`), a route data placeholder (`route/page.tsx` , coming soon), and a CSV export tool (`export/page.tsx`).

Search modes:

- **Session ID mode** — enter a session ID to look up party size data for that specific event
- **Date range mode** — select a start and end date to aggregate data across all sessions in that window

Input validation (`inputValidator.ts`) runs client-side before any API call is made. Session IDs must be numeric; dates must be valid calendar dates in YYYY-MM-DD format.

Results are displayed in a `SearchResultsTable` component. The table can be reset independently of the search form.

A `ResetButton` clears the form; a separate button clears the results table.

Quick Export Tool (`export/page.tsx`) — staff select a reporting period (1 month → all time) and a data type (Survey or Route) and download a CSV directly. Route exports use `routeDataFetchService.ts` which fetches the 10 route engagement metrics from the 20 backend route-stats endpoints in parallel.

→ Route Statistics endpoints are fully specified in [ROUTE STATISTICS SPEC.md](#). The portal route browser (`route/page.tsx`) remains a placeholder — the export tool covers date-range route stats; a session-level route browser is pending.

Shared Code (`shared/`)

```
shared/  
└─ types.ts    – RouteNode, FactNode, Route interfaces
```

`types.ts` is imported by the frontend pages (`home` , `route` , `pollinator-collection`) and by the backend's `api.interfaces.ts` and `reshape.Utilites.ts` . It defines the `Route` , `RouteNode` , and `FactNode` interfaces that cross the API boundary.

`shared/data/Routes.json` has been removed — it was previously used by the frontend to look up pollinator names and spritesheet coordinates, but both pages now fetch that data from `/api/get-pollinator-names` instead. See [SHARED TYPES.md](#) for full detail.

Running the App

Development (MySQL/MariaDB):

```
docker compose -f docker-compose.dev.yml up
```

Development (PostgreSQL):

```
docker compose -f docker-compose.dev.pg.yml up
```

Production (PostgreSQL, recommended):

```
cp .env.pg.example .env.pg # fill in credentials  
docker compose -f docker-compose.prod.pg.yml up
```

The dev stacks mount source files as volumes and use `ts-node` with `--watch` for hot reload. The prod stacks build a multi-stage distroless image for the backend.

→ See [backend-flow.md](#) § Docker & Deployment for the full service startup order and config details. → See [./MYSQL_LICENSE_ISSUE.md](#) for why PostgreSQL is recommended for non-academic deployment.

Not tracked by git — update this file whenever pages, routes, or major system relationships change.